



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Network Security

Case 2

Supervisor: Prof. Christos Xenakis

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

01/11/2025

Abstract

Packet sniffing at a low level using raw sockets on Kali Linux (VMware), with the network card in promiscuous mode. Packets are collected in a pcap file and graphically analyzed with Wireshark.

Contents

Introduction.....	1
Practical piece of work.....	2
Bibliography	8

Introduction

In this exercise we deal with packet sniffing at a low level using raw sockets. A socket is generally a communication mechanism that the operating system uses for applications to exchange data over a network. Raw sockets are a special type of socket that allows the application to directly access network packets, without the normal processing of the protocol by the kernel. In other words, raw sockets bypass standard kernel filtering and interfere with the process of sending packets to applications. Thus, in the first phase the packets are not filtered or separated for delivery to UDP/TCP sockets, and we can see all the bytes and headers as received by the network card. In this way we gain access to information that we would not otherwise see easily.

A network card (NIC) is the physical device that connects the computer to the network, and each NIC has a unique MAC address. The NIC works in different modes: promiscuous mode (in wired and wireless networks) where it reads all frames passing through the interface (i.e. the network card to the operating system), and monitor mode (in wireless networks only) where it reads frames on the specific wireless channel (i.e. at the frequency that the network card transmits/receives). For the exercise, we will activate the promiscuous mode, so that the raw socket receives all the traffic of the interface and we can analyze packets.

The VM sees a virtual NIC managed by the host. This virtual card is not the same as the real physical Wi-Fi card. So, even if the host is connected wirelessly, the VM does not have direct access to Wi-Fi traffic and only sees what the host provides to it depending on the mode (NAT or Bridged).

- NAT: The VM shares the network output with the host and gets an internal IP. External traffic is displayed with the host's IP. Practically, the VM mainly sees its own traffic, which limits the ability to sniffing the traffic of other devices in the LAN.
- Bridged: The VM behaves like a separate device on the local network (it takes IP from the router) and can exchange packets directly with other devices on the LAN. This allows for wider sniffing, but exposes the VM to the local network and carries greater risk and legal liabilities.

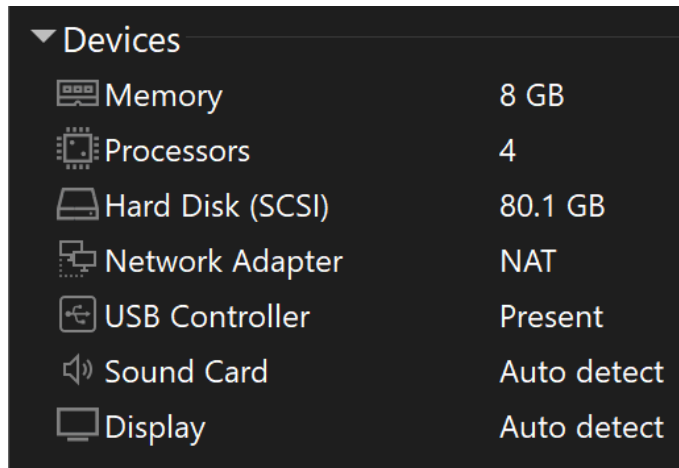
Practical piece of work

Introduction:

We will run the exercise on Kali Linux (kali-linux-2025.3-vmware-amd64) through VMware Workstation Pro (latest version) because we need root privileges to work the raw sockets and to change the card mode (promiscuous). For isolation and security reasons, we choose NAT as the default. With this setting, the VM is not directly exposed to the local network and we mainly see the traffic that the VM itself generates/receives, without accidentally collecting traffic from other LAN devices. If we need motion analysis of other devices, then we will choose Bridged or use a physical Wi-Fi adapter in monitor mode. In the VM we will also run Wireshark in parallel for graphical analysis and filter application.

Initiating Procedures:

From the characteristics of the resources we have available for this OS, we notice that the network adapter is in NAT. So we will not make any modification to this setting.

A screenshot of the 'Devices' window in VMware Workstation. The window has a dark background and a list of hardware components for a virtual machine. Each item has a small icon to its left and its configuration value to its right. The items are: Memory (8 GB), Processors (4), Hard Disk (SCSI) (80.1 GB), Network Adapter (NAT), USB Controller (Present), Sound Card (Auto detect), and Display (Auto detect).

▼ Devices		
	Memory	8 GB
	Processors	4
	Hard Disk (SCSI)	80.1 GB
	Network Adapter	NAT
	USB Controller	Present
	Sound Card	Auto detect
	Display	Auto detect

```

kali@kali -
Session Actions Edit View Help
(kali@kali)-[~]
$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
3: br-480f97c7b15a: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
    link/ether 02:42:b5:bf:19:c8 brd ff:ff:ff:ff:ff:ff
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
    link/ether 02:42:d8:55:b8:0c brd ff:ff:ff:ff:ff:ff

(kali@kali)-[~]
$ ip -br link
lo          UNKNOWN          00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0        UP                00:0c:29:fe:e8:b5 <BROADCAST,MULTICAST,UP,LOWER_UP>
br-480f97c7b15a DOWN          02:42:b5:bf:19:c8 <NO-CARRIER,BROADCAST,MULTICAST,UP>
docker0     DOWN            02:42:d8:55:b8:0c <NO-CARRIER,BROADCAST,MULTICAST,UP>

(kali@kali)-[~]
$

```

We notice that either with the **ip link show** command or with the **ip -br link** command we control our interface. The interface we will use in the exercise is **eth0** because it is the only one that is active (UP) and corresponds to the virtual card given to us by VMware (via NAT).

```

(kali@kali)-[~/Downloads]
$ ip link show eth0 | grep PROMISC
2: eth0: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff

(kali@kali)-[~/Downloads]
$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff

(kali@kali)-[~/Downloads]
$

```

The next step is to check if the promiscuous mode is active. With the command **ip link show eth0 | grep PROMISC** (the same is done with the command **ip link show eth0**) we can check if the specific mode is active. If it is not, we do it with the command **sudo ip link set eth0 promisc on** (and respectively to disable it it is the same command but ending in **off** instead of **on**).

```

kali@kali -
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo apt install libpcap-dev -y
[sudo] password for kali:
The following packages were automatically installed and are no longer required:
  anansi-common libmongoc-1.0-0t64 libtheoradec1 libyelp0 python3-kismetcapturefreshlabsrigbee samba-ad-dc
  libdbus-1-2 libbson-1.0-0t64 libtheoradec1 libyelp0 python3-blumpy python3-kismetcapturefreshlabsrigbee samba-ad-provision
  libbson-1.0-0t64 libbson-1.0-0t64 libtheoradec1 libyelp0 python3-click-plugins python3-kismetcapturefreshlabsrigbee samba-dsdb-modules
  libjs-jquery-ui libportsmidia libx264-164 python3-gpg python3-kismetcapturefreshlabsrigbee python3-kismetcapturefreshlabsrigbee
  libjs-underscore libx264-164 python3-kismetcapturefreshlabsrigbee python3-kismetcapturefreshlabsrigbee python3-kismetcapturefreshlabsrigbee
Use 'sudo apt autoremove' to remove them.

Installing:
  libpcap-dev

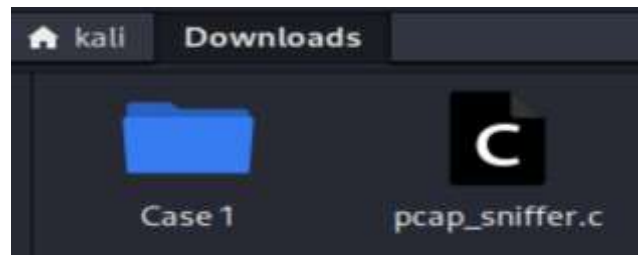
Installing dependencies:
  libcap-dev libdbus-1-dev libpcap0.8-dev libpkgconf3 libsystemd-dev pkgconf pkgconf-bin

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1
  Download size: 2,478 kB
  Space needed: 10.9 MB / 66.7 GB available

Get:1 http://kali.org/kali kali-rolling/main amd64 libcap-dev amd64 1:2.75-18+b1 [264 kB]
Get:2 http://kali.org/kali kali-rolling/main amd64 libsystemd-dev amd64 255-1 [1,309 kB]
Get:3 http://ftp.cc.uec.gr/mirrors/linux/kali/kali kali-rolling/main amd64 libpkgconf3 amd64 1.8.1-4 [36.4 kB]
Get:4 http://ftp.cc.uec.gr/mirrors/linux/kali/kali kali-rolling/main amd64 pkgconf-bin amd64 1.8.1-4 [30.2 kB]
Get:5 http://kali.org/kali kali-rolling/main amd64 pkgconf amd64 1.8.1-4 [26.2 kB]
Get:6 http://kali.org/kali kali-rolling/main amd64 libdbus-1-dev amd64 1.16.2-2 [215 kB]
Get:7 http://kali.org/kali kali-rolling/main amd64 libpcap0.8-dev amd64 1.10.5-2 [264 kB]
Get:8 http://kali.org/kali kali-rolling/main amd64 libpcap-dev amd64 1.10.5-2 [31.8 kB]
Fetched 2,478 kB in 2s (1,411 kB/s)
Selecting previously unselected package libcap-dev:amd64.
(Reading database ... 429077 files and directories currently installed.)
Preparing to unpack .../0-libcap-dev_1:2.75-18+b1_amd64.deb ...
Unpacking libcap-dev:amd64 (1:2.75-18+b1) ...
Selecting previously unselected package libsystemd-dev:amd64.
Preparing to unpack .../1-libsystemd-dev_255-1_amd64.deb ...
Unpacking libsystemd-dev:amd64 (255-1) ...
Selecting previously unselected package libpkgconf3:amd64.
Preparing to unpack .../2-libpkgconf3_1.8.1-4_amd64.deb ...
Unpacking libpkgconf3:amd64 (1.8.1-4) ...
Selecting previously unselected package pkgconf-bin.
Preparing to unpack .../3-pkgconf-bin_1.8.1-4_amd64.deb ...
Unpacking pkgconf-bin (1.8.1-4) ...
Selecting previously unselected package pkgconf.
Preparing to unpack .../4-pkgconf_1.8.1-4_amd64.deb ...
Unpacking pkgconf (1.8.1-4) ...

```

After completing the above procedures, we will download the libpcap library. Then we will make packet sniffer in the programming language in C via the library. We made the file in pcap_sniffer.c in the Downloads folder and it includes the modified code based on that of the slide provided by the instructor.



We go to the Downloads folder where we have the file and we will compile it.

```
(kali@kali)-[~]  
$ cd ~/Downloads && ls -l *.c  
-rw-rw-r-- 1 kali kali 1619 Oct 31 10:07 pcap_sniffer.c
```

With this command, in addition to "entering" the Downloads folder, we also searched for any file ending in .c. The only file in this case was the sniffer so we will proceed to compile it.


```
(kali@kali)-[~/Downloads]
$ gcc pcap_sniffer.c -o pcap_sniffer -lpcap

(kali@kali)-[~/Downloads]
$ sudo ./pcap_sniffer
Usage: ./pcap_sniffer <interface>

(kali@kali)-[~/Downloads]
$ sudo ./pcap_sniffer eth0
Capturing packets on eth0... Press Ctrl+C to stop.
Captured packet (42 bytes)
Captured packet (60 bytes)
Captured packet (71 bytes)
Captured packet (70 bytes)
Captured packet (122 bytes)
Captured packet (74 bytes)
Captured packet (376 bytes)
Captured packet (74 bytes)
Captured packet (74 bytes)
Captured packet (115 bytes)
Captured packet (66 bytes)
Captured packet (79 bytes)
Captured packet (1294 bytes)
Captured packet (933 bytes)
Captured packet (74 bytes)
Captured packet (74 bytes)
Captured packet (1034 bytes)
Captured packet (66 bytes)
Captured packet (80 bytes)
Captured packet (70 bytes)
^C
Stopping capture and closing files ...

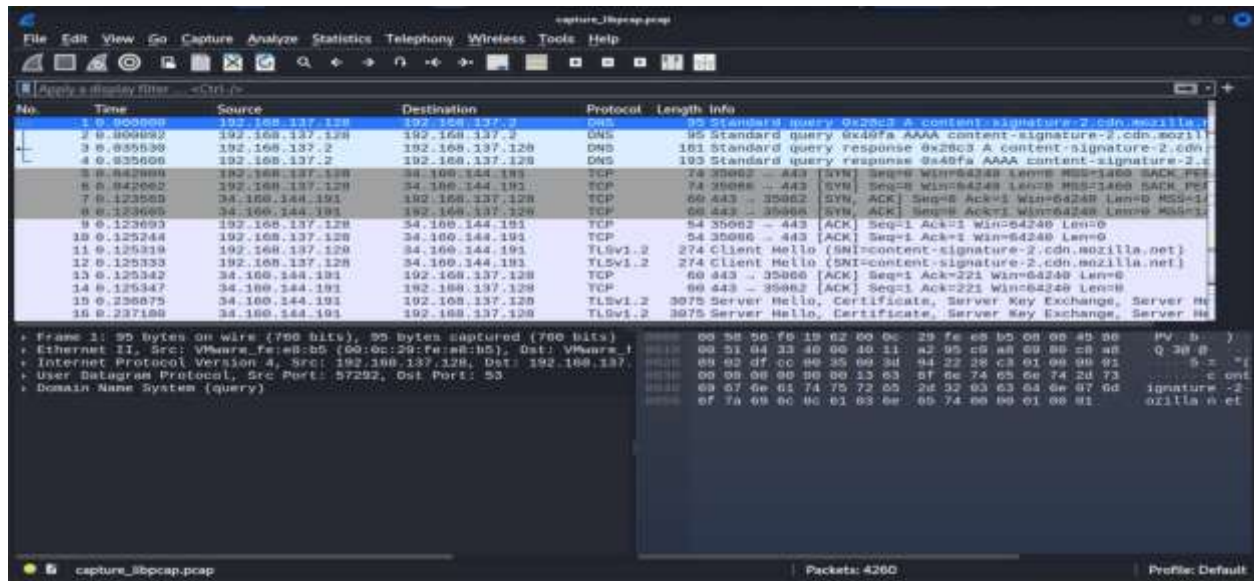
(kali@kali)-[~/Downloads]
$
```

After compiling the sniffer with the command `gcc pcap_sniffer.c -o pcap_sniffer -lpcap` through the library and making sure that it will not have any problems during compilation, we run the sniffer file and put the port that is open as the interface (`sudo ./pcap_sniffer eth0`). This is how the recording of the packets starts and we can stop the process by pressing the ctrl + C buttons on the keyboard. Completing the above process, The file (**capture_libpcap.pcap**) that we will open via Wireshark is created and includes all the information about the packages. We will open the file with the wireshark command <name of the file> (**wireshark capture_libpcap.pcap**).



wireshark will open and display all the packets.

Then we can select the Statistics to draw the first conclusions:



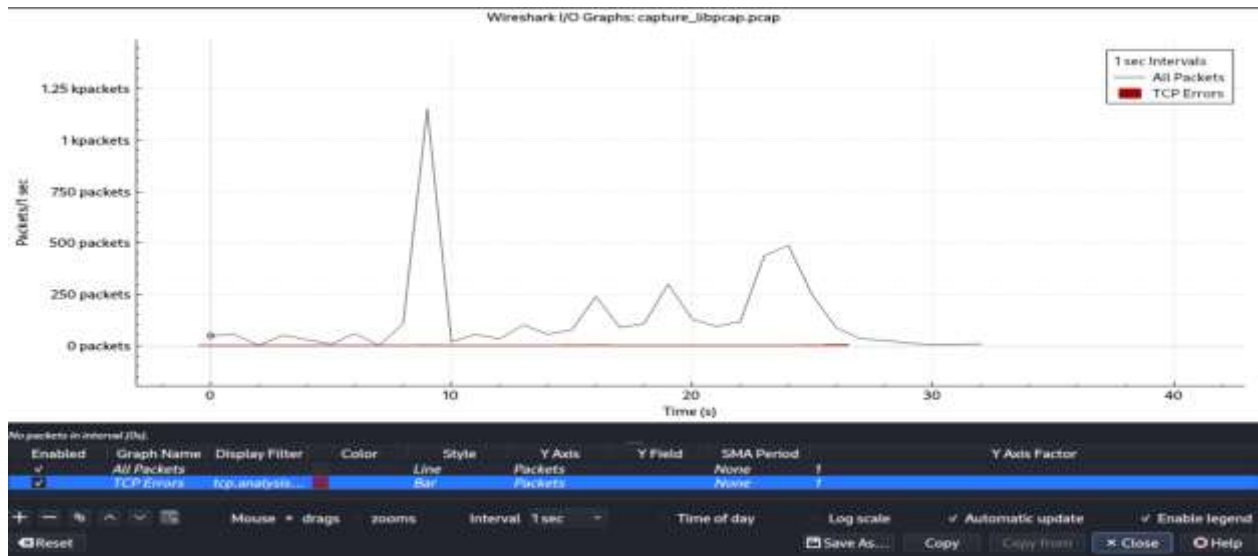
Wireshark - Protocol Hierarchy Statistics - capture_libpcap.pcap

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes
- Frame	100.0	4260	100.0	2826015	698 k	0	0
- Ethernet	100.0	4260	2.3	63622	15 k	0	0
- Internet Protocol Version 6	0.2	7	0.0	280	69	0	0
- User Datagram Protocol	0.2	7	0.0	56	13	0	0
- Multicast Domain Name System	0.2	7	0.0	875	216	7	875
- Internet Protocol Version 4	99.5	4237	3.0	84740	20 k	0	0
- User Datagram Protocol	56.4	2403	0.7	19224	4,749	0	0
- QUIC IETF	48.2	2052	61.5	1738829	429 k	2052	1675620
- Multicast Domain Name System	0.2	7	0.0	875	216	7	875
- Domain Name System	8.1	344	1.0	29116	7,193	344	29116
- Transmission Control Protocol	42.5	1811	1.3	37996	9,387	1131	24396
- Transport Layer Security	15.9	678	31.2	881202	217 k	678	822203
- Hypertext Transfer Protocol	0.0	2	0.0	518	127	1	310
- Line-based text data	0.0	1	0.0	8	1	1	8
- Internet Control Message Protocol	0.5	23	0.3	8017	1,980	23	8017
- Address Resolution Protocol	0.4	16	0.0	448	110	16	448

Protocol changes have been reverted.

Close Copy Protocols Help

- From the analysis of the hierarchy of protocols, it can be observed that the majority of traffic is based on IPv4, with UDP (56.4%) and TCP (42.5%) being the dominant transport protocols.



- From the flow graph (I/O Graph) there is a change in the number of packets per second, with a sharp peak at about 10 seconds and smaller increases thereafter. This increase indicates periods of intense network activity, possibly when loading data or initiating new connections. No significant TCP errors are detected, indicating smooth operation and stable data transfer.

Conclusion:

The exercise demonstrated the practical operation of a packet sniffer at a low level and the effectiveness of Wireshark in analyzing and visualizing network traffic.

Bibliography

Moon, S. (2020, July 26). How to code raw sockets in C on Linux - BinaryTides. Retrieved from <https://www.binarytides.com/raw-sockets-c-code-linux/>

nospaceships/raw-socket-sniffer: Packet capture on Windows without a kernel driver. (n.d.). Retrieved from <https://github.com/nospaceships/raw-socket-sniffer>

packet(7) - Linux manual page. (n.d.). Retrieved from <https://man7.org/linux/man-pages/man7/packet.7.html>

Harrichael. (2014, July 25). C raw socket receive or sniff incoming packets only. Retrieved from <https://stackoverflow.com/questions/24964586/c-raw-socket-receive-or-sniff-incoming-packets-only>

Raw Socket programming - Efficient Packet Sniffer - Applications / Programming - Unix Linux Community. (2013, June 26). Retrieved from <https://community.unix.com/t/raw-socket-programming-efficient-packet-sniffer/331041/10>

Ssaf. (n.d.). How raw sockets work? Retrieved from <https://superuser.com/questions/1678667/how-raw-sockets-work>

GeeksforGeeks. (2025, July 15). Kali Linux sniffing and spoofing. Retrieved from <https://www.geeksforgeeks.org/linux-unix/kali-linux-sniffing-and-spoofing/>

ChatGPT. (n.d.). Retrieved from <https://chatgpt.com/>